Routing Guide

Here is the expanded and formatted Markdown note for your static routing topology, complete with the explanations for the commands to match your previous study guides.

---

tags:

- networking
- routing
- linux
- static-routes

aliases:

- Static Routing Guide
- Square Topology Routing

Static Routing for Network Connectivity

Topic: Routing

To ensure every host in a square network topology can successfully ping every other host, each router must be configured with a static route for the two networks it is not physically connected to.

1. The Network Layout

Based on a standard IP scheme, the four networks (Net 1-4) connecting the routers are defined as follows:

- **Net 1**: `192.168.1.0/24` (Connects R1 and R2)
- **Net 2**: `192.168.2.0/24` (Connects R2 and R3)
- **Net 3**: `192.168.3.0/24` (Connects R3 and R4)
- **Net 4**: `192.168.4.0/24` (Connects R4 and R1)

---

2. The Prerequisite: Enabling IP Forwarding

Before adding any routes, every single router must have IP forwarding enabled.

The Command:

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
```

🧠 **Why are we doing this?** By default, a Linux machine acts as a *host*, meaning it drops any incoming network packets that aren't specifically addressed to its own IP. Changing this kernel parameter to `1` tells the machine to act as a *router*, allowing it to accept packets on one interface and forward them out of another interface toward their final destination.

---

3. Router 1 (R1) Configuration

- **Physically Connected to**: Net 1 and Net 4.
- **Needs routes to**: Net 2 and Net 3.

The Commands:

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo ip route add 192.168.2.0/24 via 192.168.1.2
sudo ip route add 192.168.3.0/24 via 192.168.4.4
```

🧠 **Why `via`?** The `via` keyword specifies the "next hop" IP address. For R1 to send traffic to Net 2, it must hand the packet off to R2's interface (`192.168.1.2`). To reach Net 3, it hands the packet to R4's interface (`192.168.4.4`).

Resulting Routing Table (`ip route`):

Plaintext

```
192.168.1.0/24 dev eth0 proto kernel scope link
192.168.4.0/24 dev eth1 proto kernel scope link
192.168.2.0/24 via 192.168.1.2 dev eth0
192.168.3.0/24 via 192.168.4.4 dev eth1
```

---

4. Router 2 (R2) Configuration

- **Physically Connected to**: Net 1 and Net 2.
- **Needs routes to**: Net 3 and Net 4.

The Commands:

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo ip route add 192.168.3.0/24 via 192.168.2.3
sudo ip route add 192.168.4.0/24 via 192.168.1.1
```

Resulting Routing Table (`ip route`):

Plaintext

```
192.168.1.0/24 dev eth0 proto kernel scope link
192.168.2.0/24 dev eth1 proto kernel scope link
192.168.3.0/24 via 192.168.2.3 dev eth1
192.168.4.0/24 via 192.168.1.1 dev eth0
```

---

5. Router 3 (R3) Configuration

- **Physically Connected to**: Net 2 and Net 3.
- **Needs routes to**: Net 4 and Net 1.

The Commands:

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo ip route add 192.168.4.0/24 via 192.168.3.4
sudo ip route add 192.168.1.0/24 via 192.168.2.2
```

Resulting Routing Table (`ip route`):

Plaintext

```
192.168.2.0/24 dev eth0 proto kernel scope link
192.168.3.0/24 dev eth1 proto kernel scope link
192.168.4.0/24 via 192.168.3.4 dev eth1
192.168.1.0/24 via 192.168.2.2 dev eth0
```

---

6. Router 4 (R4) Configuration

- **Physically Connected to**: Net 3 and Net 4.

- **Needs routes to**: Net 1 and Net 2.

The Commands:

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo ip route add 192.168.1.0/24 via 192.168.4.1
sudo ip route add 192.168.2.0/24 via 192.168.3.3
```

Resulting Routing Table (`ip route`):

Plaintext

```
192.168.3.0/24 dev eth0 proto kernel scope link
192.168.4.0/24 dev eth1 proto kernel scope link
192.168.1.0/24 via 192.168.4.1 dev eth1
192.168.2.0/24 via 192.168.3.3 dev eth0
```

---

7. Critical Note for the Exam: The Hosts

Even if all four routers are configured perfectly, your lab will fail if the end-user machines aren't set up correctly.

The Rule: If the Hosts (the PCs connected to these networks) cannot ping each other, you must check their Default Gateway.

A host on Net 1 must have its gateway set to the IP of either R1 or R2, depending on whichever router it is physically plugged into.

The Command (Run on the Host PC):

Bash

```
sudo ip route add default via 192.168.1.1
```

🧠 **Why are we doing this?**

A host computer doesn't know the layout of the square topology. Setting the `default` route tells the PC: *"If you are trying to reach an IP address that isn't on your local switch, hand the packet to the router at `192.168.1.1` and let the router figure it out."* Without this, the ping never even leaves the host machine.

Would you like to review how to make these `ip route` commands persistent so they don't disappear if you accidentally reboot one of your lab machines?